

D  ployer app_api sur son propre EC2

Instance unique servant trois r  les : reverse proxy TLS (Caddy), interface utilisateur (Streamlit) et API de pr  diction (FastAPI) pour NappeCast    c'est aussi le seul point d'entr  e public vers Airflow et MLflow.

BUG ACTIF    SECRETS ET CREDENTIALS NEUTRALIS  S DANS LE CONTENEUR API

Dans `docker/app_api/docker-compose.yml`, le service `api` d  clare un bloc `environment:` redondant avec des `${VAR}` :

YAML    EXTRAIT FAUTIF

```
api:
  environment:
    MLFLOW_TRACKING_URI: ${MLFLOW_TRACKING_URI}
    PIPELINE_SECRET: ${PIPELINE_SECRET}
    AWS_ACCESS_KEY_ID: ${AWS_ACCESS_KEY_ID}
    AWS_SECRET_ACCESS_KEY: ${AWS_SECRET_ACCESS_KEY}
```

Ces `${VAR}` sont interpol  es par **Compose lui-m  me**, qui ne les r  sout qu'   partir d'un fichier litt  ralement nomm   `.env`    absent ici. Elles valent donc **cha  ne vide**, et comme `environment:` a priorit   sur `env_file:` pour une m  me cl  , ce bloc   crase silencieusement les bonnes valeurs pourtant fournies par `env_file: [appapi.env, .env.runtime]`. R  sultat : l'API tourne sans `PIPELINE_SECRET` ni credentials AWS, sans erreur visible au d  marrage. **Correctif : supprimer enti  rement ce bloc `environment:`    `env_file` suffit.**

CL   AWS STATIQUE ET MOT DE PASSE RDS EN CLAIR

`docker/app_api/aws_create_secret.sh` pousse dans `appapi/prod-latest` : la m  me paire de cl  s AWS statiques que MLflow, le `PIPELINE_SECRET`, et l'URI de connexion RDS avec le mot de passe du compte ma  tre Postgres en clair. Voir    Gestion des IAM    pour le plan de remplacement de la cl   statique par un r  le d'instance. Le mot de passe RDS m  rite un traitement    part : **tournez-le** (`aws rds modify-db-instance --master-user-password ...`) d  s que ce document circule, puis mettez    jour uniquement la valeur dans Secrets Manager    inutile de toucher au code.

- 1 Le secret
- 2 R  le IAM EC2
- 3 docker-compose
- 4 deploy.sh
- 5 Caddyfile
- 6 Workflow CI/CD
- 7 Checklist

1 Secret `appapi/prod-latest`

CL��	R��LE
<code>PIPELINE_SECRET</code>	Jeton partag�� avec les DAGs Airflow (<code>Variable.get("pipeline_secret")</code>) pour authentifier les appels DAG �� API
<code>NAPPECAST_BACKEND_STORE_URI</code>	Cha��ne de connexion RDS Postgres (contient le mot de passe �� �� tourner, voir alerte ci-dessus)
<code>AWS_ACCESS_KEY_ID</code> / <code>AWS_SECRET_ACCESS_KEY</code>	�� retirer �� remplacer par un r��le d'instance (�� Gestion des IAM ��, ��tape 3)

2 R  le IAM de l'instance

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "secretsmanager:GetSecretValue",
      "Resource": "arn:aws:secretsmanager:eu-west-3:844099234486:secret:appapi/prod-latest-*"
    },
    { "Effect": "Allow", "Action": "ecr:GetAuthorizationToken", "Resource": "*" },
    {
      "Effect": "Allow",
      "Action": ["ecr:BatchGetImage", "ecr:GetDownloadUrlForLayer"],
      "Resource": [
        "arn:aws:ecr:eu-west-3:844099234486:repository/api",
        "arn:aws:ecr:eu-west-3:844099234486:repository/streamlit"
      ]
    }
  ]
}
```

Pas de permission S3 n  cessaire ici une fois la cl   statique retir  e : l'acc  s aux artefacts MLflow depuis l'API passe par le tracking server MLflow, pas par un acc  s S3 direct       v  rifier dans le code de `src/api` si un acc  s S3 direct existe malgr   tout, auquel cas ajouter la m  me permission que dans le r  le MLflow.

3 docker-compose.yml    trois services

YAML   " /OPT/APP_API/DOCKER-COMPOSE.YML (CORRIG  )

```
services:
  caddy:
    image: caddy:2
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data
      - caddy_config:/config
    restart: unless-stopped

  streamlit:
    image: 844099234486.dkr.ecr.eu-west-3.amazonaws.com/streamlit:latest
    env_file:
      - appapi.env
      - .env.runtime
    restart: unless-stopped

  api:
    image: 844099234486.dkr.ecr.eu-west-3.amazonaws.com/api:latest
    env_file:
      - appapi.env
      - .env.runtime
    restart: unless-stopped
# (bloc "environment:" supprim      voir alerte en t  te de document)
```

```
volumes:
  caddy_data:
  caddy_config:
```

appapi.env (non-secret, versionnée) :

```
DOTENV_PATH= DOCKER/APP_API/APPAPI.ENV
MLFLOW_TRACKING_URI="http://172.31.32.79:5000/"
MLFLOW_EXPERIMENT_NAME="nappe-cast"
AWS_DEFAULT_REGION=eu-west-3
API_URL="http://172.31.41.22:8000"
```

POURQUOI STREAMLIT N'A PAS LE MÊME BUG

Le service streamlit n'a pas de bloc environment redondant car seul api est concerné. C'est cohérent : Streamlit n'a besoin que de API_URL (non secret) pour appeler FastAPI en interne sur le réseau Docker.

4 deploy.sh

```
BASH_PATH="/OPT/APP_API/DEPLOY.SH"
#!/usr/bin/env bash
set -euo pipefail

APP_DIR=/opt/app_api
ENV_FILE="$APP_DIR/.env.runtime"
SECRET_ID="appapi/prod-latest"
REGION="eu-west-3"
ECR_REGISTRY="844099234486.dkr.ecr.eu-west-3.amazonaws.com"

umask 077
aws secretsmanager get-secret-value \
  --secret-id "$SECRET_ID" --region "$REGION" \
  --query SecretString --output text \
  | jq -r 'to_entries[] | "\(.key)=\(.value)"' > "$ENV_FILE"

chmod 600 "$ENV_FILE"
chown appuser:appuser "$ENV_FILE"

aws ecr get-login-password --region "$REGION" \
  | docker login --username AWS --password-stdin "$ECR_REGISTRY"

cd "$APP_DIR"
docker compose pull
docker compose --env-file "$ENV_FILE" up -d
docker image prune -f
```

LE SCP DE LA CI NE COPIE PAS APPAPI.ENV

Dans ci-app_api.yml, l'écriture SCP ne liste que docker/app_api/docker-compose.yml,docker/app_api/deploy.sh car appapi.env n'est jamais transférée. Contrairement à Airflow et MLflow (où le fichier .env non secret est bien inclus dans le SCP), le compose de app_api référence appapi.env via env_file sans qu'il n'arrive jamais sur l'instance au-delà du tout premier déploiement manuel. Tant que le

fichier a été placé une fois à la main dans `/opt/app_api/`, les déploiements suivants fonctionnent silencieusement avec cette version figée. Toute modification de `appapi.env` dans le repo ne sera **jamais appliquée** par la CI. Corrigez la ligne `source:` du SCP pour inclure `docker/app_api/appapi.env`, comme pour les deux autres services.

5 Caddyfile est le seul point d'entrée public

Cette instance termine le TLS (via Cloudflare) pour les trois domaines et relaie en interne. C'est la seule porte d'entrée et seuil pour Airflow et MLflow, qui ne sont eux-mêmes jamais exposés directement.

DOMAINE	BACKEND	ACCÈS
<code>nappecast.mapflowing.com</code>	<code>streamlit:8501</code> (seuil Docker interne)	Public
<code>airflow.mapflowing.com</code>	IP privée VPC de l'instance Airflow, port 8080	Allowlist IP
<code>mlflow.mapflowing.com</code>	IP privée VPC de l'instance MLflow, port 5000	Allowlist IP

POINTS D'AJUSTEMENT BIEN FAITS

- `trusted_proxies` / `client_ip_headers` configurés pour Cloudflare. L'allowlist IP filtre sur la vraie IP client, pas sur celle de Cloudflare.
- L'allowlist (`allowlist` snippet réutilisé) n'est appliquée qu'aux deux domaines internes. L'app publique reste ouverte comme prévu.

DÉPENDANCE À DES IP PRIVÉES NON-ELASTIC

Les deux backends sont référencés par IP privée VPC brute (`172.31.x.x`). Un redémarrage ou remplacement des instances Airflow/MLflow peut faire changer cette IP et casser silencieusement le proxy (erreurs 502 sans lien évident avec une modif de ce fichier). Envisagez des IP privées Elastic, ou une zone DNS privée de Route 53 interne, pour découpler ce fichier du cycle de vie des instances backend.

6 Workflow CI/CD (`ci-app_api.yml`)

Seul des trois workflows à comporter un job `test` (pytest, Python 3.13) qui doit réussir (`needs: test`) avant tout build/déploiement. Bonne pratique à répliquer sur les deux autres services si des tests y sont ajoutés un jour.

SECRET GITHUB	USAGE
<code>APIAPP_SG_ID</code>	Security group dont le port 22 est ouvert/fermé le temps du déploiement
<code>APIAPP_EC2_HOST</code> / <code>_USER</code> / <code>_SSH_KEY</code>	Connexion SCP/SSH à cette instance

DÉCLENCHEURS AUTOMATIQUES DÉACTIVÉS

Les triggers `push` / `pull_request` sont actuellement commentés. Seul `workflow_dispatch` (déclenchement manuel) est actif. Un correctif poussé sur `src/api` ou `src/app` ne sera donc **pas** déployé automatiquement, contrairement à ce qu'on pourrait attendre en miroir de `cd-airflow.yml` / `cd-mlflow.yml` qui se déclenchent sur `push`. À réactiver consciemment (avec le filtre `paths:`

approprié) une fois que ce document et les correctifs ci-dessus auront été appliqués.

7 Checklist

✓ Bloc `environment:` du service `api` supprimé du `docker-compose.yml`.

✓ `appapi.env` ajouté au SCP de `ci-app_api.yml`.

✓ Mot de passe RDS tourné et secret mis à jour.

✓ Rôle IAM d'instance en place, clé statique retirée.

✓ Triggers `push` réactivés une fois les correctifs ci-dessus validés.

Voir « Gestion des IAM » pour les rôles, « Déploiement MLflow EC2 » et le futur document Airflow pour le détail des backends proxifiés ici.